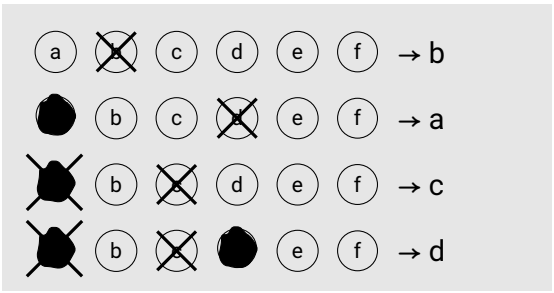
**Exercises**

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| 3 | 3 | 3 | 3 | 3 | 3 | 3 |
| 4 | 4 | 4 | 4 | 4 | 4 | 4 |
| 5 | 5 | 5 | 5 | 5 | 5 | 5 |
| 6 | 6 | 6 | 6 | 6 | 6 | 6 |
| 7 | 7 | 7 | 7 | 7 | 7 | 7 |
| 8 | 8 | 8 | 8 | 8 | 8 | 8 |
| 9 | 9 | 9 | 9 | 9 | 9 | 9 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

**Surname, First name**

**Algorithmic Design (BCS1540)**
BCS1540 Exam




Answer multiple-choice questions as shown in the example.

**Program:** BSc Computer Science
**Course code:** BCS1540
**Examiners:** Steven Chaplick & David Mestel
**Date/time:** 21/05/2025 12:00-14:00hr
**Format:** Closed book
**Allowed aids:** Pens (black or dark blue), NO calculator allowed
**Instructions to students:**

- The exam consists of 5 questions on 14 pages.
- Fill in your name and student ID number on the cover page and tick the corresponding numerals of your student number in the table (top right cover page).
- Answer every question in the reserved space below the question. **Do <u>not</u> write outside the reserved space or on the back of pages, this will not be scanned and will NOT be graded!** As a last resort if you run out of space, use the extra answer space at the end of the exam.
- *In no circumstance write on or near the QR code at the bottom of the page!*
- Ensure that you properly motivate your answers.
- Only use black or dark blue pens, and write in a readable way. Do not use pencils.
- Answers that cannot be read easily cannot be graded and may therefore lower your grade.
- If you think a question is ambiguous, or even erroneous, and you cannot ask during the exam to clarify this, explain this in detail in the space reserved for the answer to the question.
- If you have not registered for the exam, your answers will not be graded, and thus handled as invalid.
- You are not allowed to have a communication device within your reach, nor to wear or use a watch.
- You have to return all pages of the exam. You are not allowed to take any sheets, even blank, home.
- Good luck!

## Greedy Algorithms

You are operating an interstellar shipping company and need to plan the re-fueling stops of one of your cargo ships. Re-fueling requires the ship to stop at a fuel station, and you want your ship to make as few stops as possible.

Luckily the route your ship will travel is a simple single high-speed corridor, e.g., similar to a highway. There are $n$ fueling stations along this route (between where the ship will enter and exit the corridor). The stations are located at distances $0 \leq d_1 \leq d_2 \leq \ldots \leq d_n$ from where your ship will enter the corridor. The distance from where the ship enters to where the ship exists is denoted by $d^\star$.

After re-fueling the ship can travel a distance of at most $t > 0$ before running out of fuel. You may assume that no fuel is consumed when the ship:

- travels from its initial position to its starting point in the corridor
- travels from its final position in the corridor (at distance $d^\star$) to its destination
- exits the corridor at position $d_i$ to re-fuel at fueling station $i$.
- re-enters the corridor at position $d_i$ after re-fueling at station $i$.

For example, if $d^\star = 10$, $t = 3$, $n = 4$ and the stations are at distances 2, 3, 6 and 8, your ship could make stops at stations 2, 3 and 4. However, it could not makes stops at stations 1, 3 and 4 since it would run out of fuel between station 1 (at distance 2) and station 3 (at distance 6).

You need to choose a sequence of stops your ship can make, with as few stops as possible.

8p **1a** Write a greedy algorithm to solve this problem in pseudocode, and include a brief high-level English description of your algorithm's main idea(s).
Your algorithm should return an ordered list of stations $1 \leqslant i_1, i_2, \cdots, i_k \leqslant n$ such that the re-fueling stops occur in this order and $k$ is as small as possible so that the ship does not run out of fuel as it travels from its starting position to its destination.

4p **1b** Prove that the algorithm you provided is optimal.

## Divide and conquer

Apply the Master Theorem to solve each of the following recurrences. Give very brief justification for each of your answers.

4p **2a** $T(n) = 6T\left(\frac{n}{2}\right) + n$

4p **2b** $T(n) = 8T\left(\frac{n}{2}\right) + 2^n$

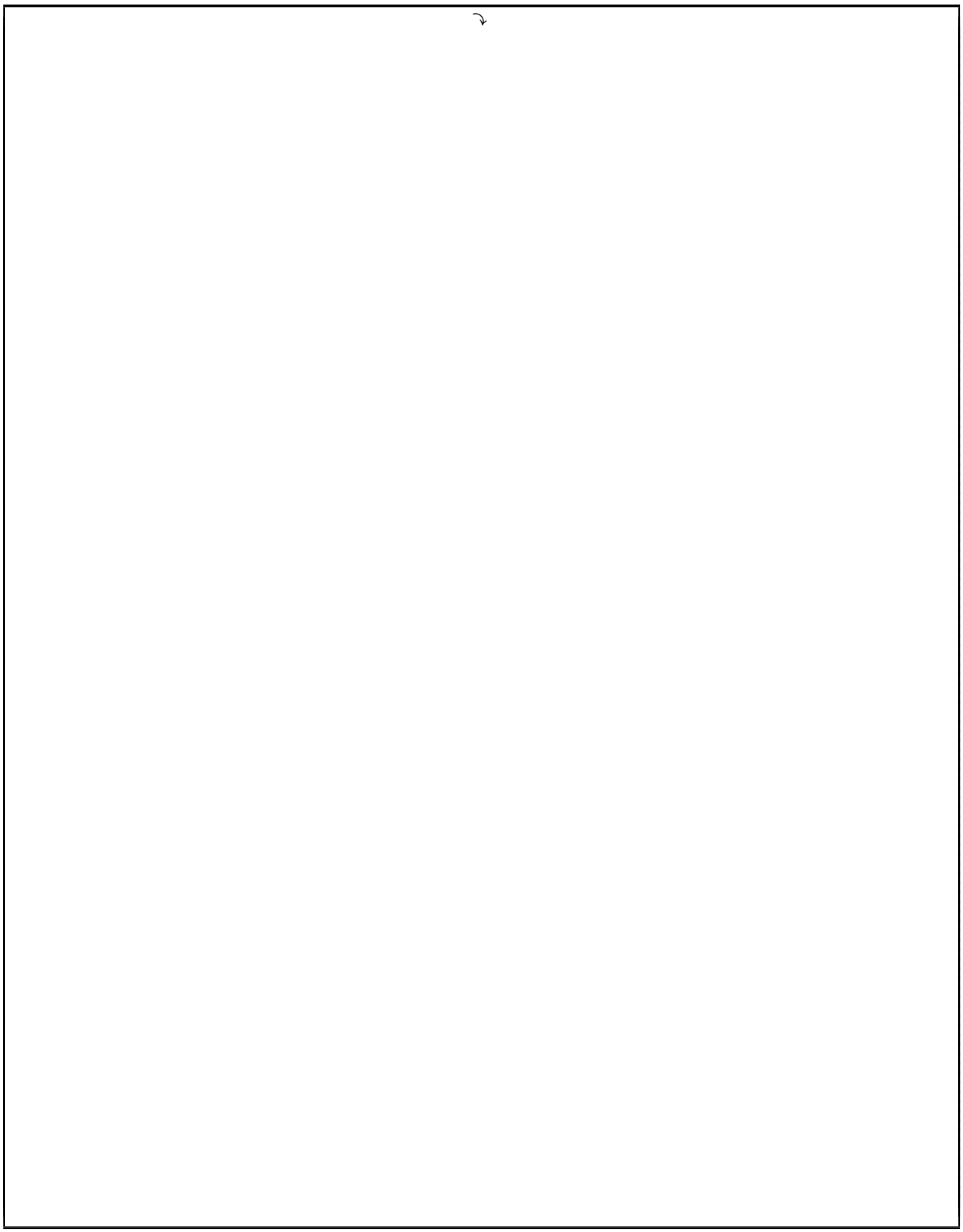
4p **2c** $T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n}$

## Dynamic Programming

Suppose you are given a length $n$ array $X$ of **integers**. For any two integer indices $i, j$, where $0 \leq i \leq j \leq n-1$, the subarray $X[i...j]$ of $X$ is the array consisting of elements $X[i], X[i+1], \ldots, X[j]$.
Your goal is to find a subarray $X[i...j]$ of $X$ whose sum is as large as possible, i.e., to find $i, j$ such that $\sum_{p=i}^{j} X[p]$ is maximized. (by brute force we can find such an $i, j$ in $O(n^3)$ time)

For example, for the array of values [−2, 1, −3, 4, −1, 2, 1, −5, 4], the subarray with the largest sum is [4, −1, 2, 1], with sum 6.

18p **3** Use dynamic programming to provide a much faster algorithm. Your answer should contain the following parts:
**Step 1**. Define a table of values. (clearly state the meaning of each table entry: the specific subproblem it corresponds to and what specific value it stores).
**Step 2**. Give a recurrence for the values in the table, and justify that your recurrence is correct (by reasoning about the recursive structure of optimal solutions).
**Step 3**. Give a bottom-up algorithm that computes the values in the table following the recurrence, and analyze the runtime of your algorithm.
**Step 4**. Give an algorithm that reconstructs an optimal solution from the table values (include an explanation of any additional information required and a brief justification that your solution is constructed correctly, as well as a brief analysis of your algorithm's runtime).



## NP-completeness and linear programming

As the newly-elected Emperor of the land of Algorithmica, you are planning to upgrade the road network of your Kingdom so that your decrees can reach your Noblemen and Noblewomen as quickly as possible.

The Kingdom consists of $n$ towns named $1, 2, \ldots, n$ (your ancestor who founded the Kingdom was not very imaginative). Town 1 is the Imperial Capital. The towns are connected in a tree by $n - 1$ roads, $r_1, r_2, \ldots, r_{n-1}$, each connecting two towns. Each road $r_i$ has a travel time $b_i$ (meaning that travelling along it takes time $b_i$), and a cost $c_i$ to upgrade it, all given as part of the input. Upgrading a road **halves** its travel time (i.e. the travel time becomes $\frac{b_i}{2}$). You have $k$ Nobles, who live at towns $N_1, N_2, \ldots, N_k$ respectively, also given as part of the input. It is always the case that lower-numbered towns are closer to the capital (i.e. the unique path from town 1 to town $i$ will pass through an increasing sequence of towns).

When you issue a decree, messengers fan out from the Imperial Capital (at town 1) along all of the roads leading out from it. Whenever a messenger reaches a new town, new messengers set out along all of the roads leading from that town, and so on until the news has reached every part of the Kingdom. For example, if $n = 4$ and the roads connect the pairs of towns (1,2), (1,3) and (3,4) and all have $b_i = 10$, the decree will reach towns 2 and 3 at time 10 and town 4 at time 20.

You have a budget $B$ for road upgrades, and you want to minimise $T_{N_1} + T_{N_2} + \cdots + T_{N_k}$, where $T_{N_i}$ is the time it takes for the decree to reach town $N_i$, after the chosen upgrades have been performed. In the example above, if $k = 2$ and the Nobles are at towns 3 and 4 and the budget $B = 10$ then the optimal choice is to upgrade the road (1,3) and then the objective function $T_{N_1} + T_{N_2} = 5 + 15 = 20$.

5p **4a** Formulate this problem as a decision problem.

2p **4b** State briefly the two conditions required for a decision problem to be NP-complete

10p **4c** Show that the decision version of this problem is NP-complete. (Hint: to show that the problem is NP-hard you may want to use a reduction from the Knapsack problem)

6p **4d** Formulate the problem as an Integer Linear Programming problem, using variables $u_1, u_2, \ldots, u_{n-1}$ and $T_1, \ldots, T_n$, where the variable $u_i$ represents whether road $i$ has been upgraded, and the variable $T_j$ represents the time it takes for the decree to reach town $T_j$ (hence in particular $T_1 = 0$).

5p **4e** State what is meant by the fractional relaxation of the ILP problem found above, and explain briefly how it can be used to solve the problem heuristically.

## Randomized Algorithms

A $3$-coloring of a graph $G$ is an assignment of colors to the vertices of $G$ where at most three colors, e.g., red, green, blue, are used. A $3$-coloring is **proper** if every edge of G is bichromatic, i.e., for every edge $e = uv$, the color of $u$ is different from the color of $v$. In the **proper 3-coloring problem**, the input is a graph $G$, and the goal is to determine whether or not $G$ has a proper 3-coloring.

4p **5a** Consider the following Las Vegas algorithm for proper 3-coloring of a given graph G.

```
Initialize the coloring by assigning every vertex to be red.
While the coloring is not proper
    for each vertex v in V(G)
        v is assigned, with equal probability, a random color among {red, green, blue}.
```

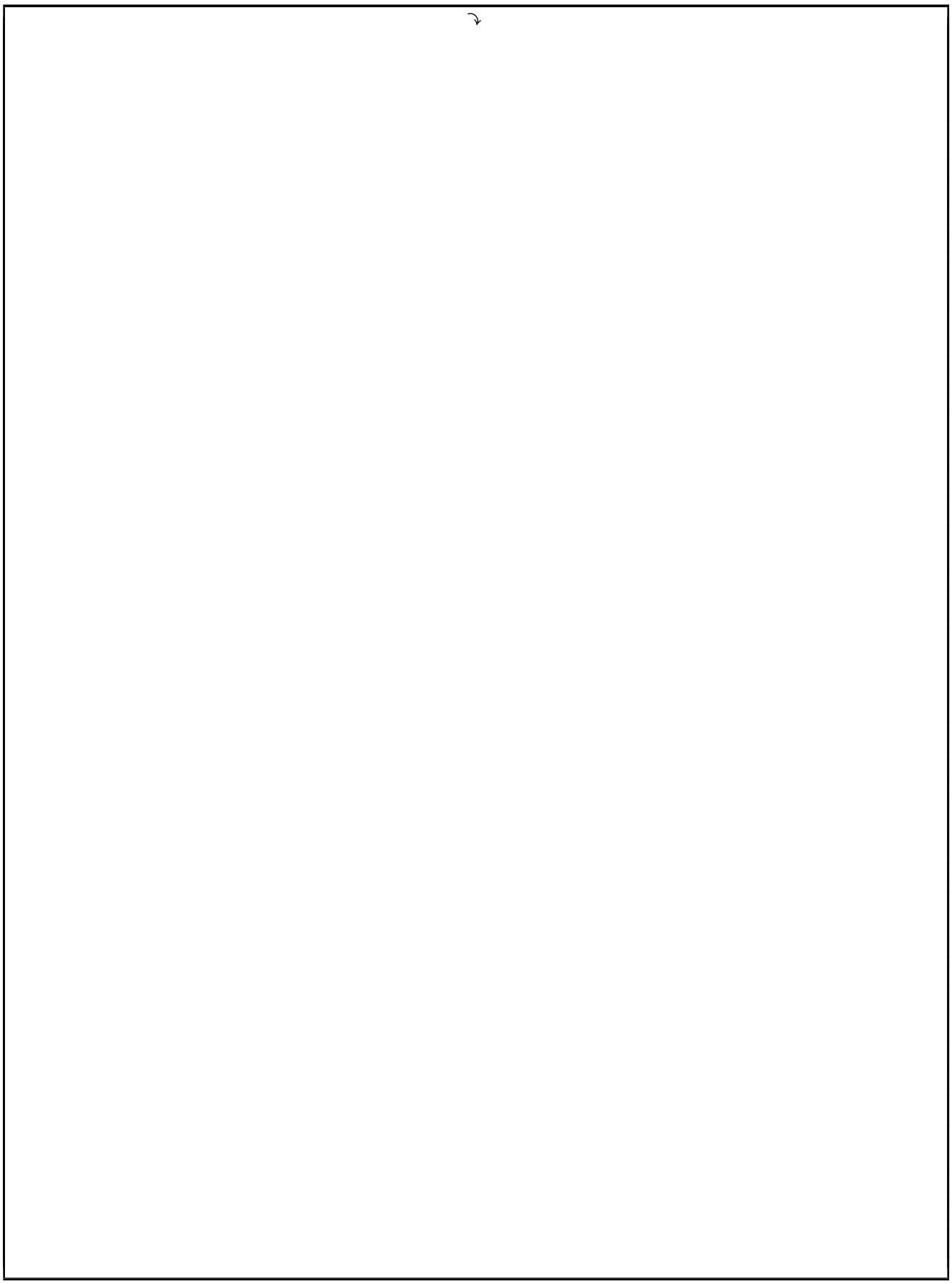
What is the expected runtime of this algorithm?

6p **5b** Provide the Monte Carlo algorithm corresponding to the Las Vegas algorithm given in part a. What is the probability that an edge is bichromatic in a coloring produced by the algorithm?

**Extra Space**

Any answers that did not fit in the space provided above can be given here. For each answer, you must clearly state the question you are answering.

**6**



This page is left blank intentionally